Prompt Workbook — CircuGrid Brief

Srujana 2026 · "Prompt to Product: Prompt for the Planet" Predictive Circular-Resource Grid for a Multi-Building Campus
Repository: https://github.com/HARDIKSINGH150206/p2p

This workbook records how the product was specified, built, tested and shipped with a generative-AI coding agent, from the first data audit to deployment. It is written as an engineering log: each entry states the intent, the prompt, what the model produced, how we verified it, and what we had to correct.

---

0. How to read this document (and what is and is not verbatim)

Each prompt entry has up to five parts:

Part	Meaning
As typed	The original message, quoted and trimmed. Typos and capitals are kept. This is the authoritative record of what was asked.
Engineered prompt	The same request restated the way a staff-level AI engineer would write it: explicit role, inputs, constraints, acceptance criteria. **These are reconstructions written afterwards for the workbook, not the literal text sent.** The constraints in them were real: they came from the project rules (`CLAUDE.md`), the evaluator prompt, or the corrections we gave in conversation.
Model output	Files and behaviour the agent actually produced, from the commit history.
Verification	How we checked it. Numbers quoted here are copied from real runs of the code in this repository.
Corrections	What was wrong, who caught it, and what changed.

Nothing in this workbook is a projected or invented metric. Where a target was missed, it says so (§6).

Times are approximate wall-clock IST on 24 Oct 2026, reconstructed from commit and session order; the hackathon deadline was 14:00.

---

1. AI tools used

Tool	Role	Where it ran
Claude Code (Claude Sonnet 5.5)	Primary coding agent: data audit, ML modules, API, frontend, tests, documentation, git	Terminal, repo working tree
Claude subagents (fresh context each)	Independent *ideator* and *evaluator* for the idea-selection loop, 3 rounds	Spawned by the orchestrating session
Headless Chrome via DevTools protocol	Interaction testing and screenshots of the UI (not an AI tool; used to verify AI-written code)	Local
scikit-learn (Ridge, RandomForest)	The *runtime* intelligence in the product	Local CPU

Runtime AI in the shipped product: classical ML only (ridge regression, random forest, robust-statistics anomaly detection, rule-based attribution). The project plan allowed an optional LLM for wording recommendation text with a template fallback; **that LLM path was not built**, so the product has no external API dependency and needs no key.

1a. Public chat share links

The competition asks for verified public share links to the AI chats. They can only be generated from the chat interface by the account owner, so they are **not** reproduced here and none has been invented.

Session	Tool	Public share link
Main build session (data audit → deployment)	Claude Code	TODO (team): paste the public share link
Ideator subagent, rounds 1–3	Claude subagent	TODO (team): paste the link, or state that it was part of the main session
Evaluator subagent, rounds 1–3	Claude subagent	TODO (team): paste the link, or state that it was part of the main session

---

2. Prompting strategy

We treated the agent as a fast but fallible junior engineer who needs a spec, a contract and a reviewer. Five techniques carried the project.

1. **Constitution first.** A single `CLAUDE.md` fixed the non-negotiables before any code: deadline, scoring rubric, "every number computed from the dataset", "a simple working product beats an ambitious broken one", "update the workbook after every task", never touch raw data, never commit secrets. Every later prompt could be short because the rules were already loaded.
2. **Generator–critic loop for the idea.** Instead of picking an idea by taste, we ran a *product-manager* prompt against a separate *judge* prompt in fresh subagents (no shared reasoning). The judge scored against the real five-criterion rubric, listed critical failures, and returned a hand-off block that the next round consumed verbatim. Scores: **7.2 → 8.0 → 8.3** (never reached the 8.5 gate; we reported that honestly and built from the best version).
3. **Contract-first parallelism.** Before splitting work between two people, we wrote `docs/CONTRACTS.md`: function signatures and JSON shapes per module. Each module could then be built, and the API stubbed, independently.
4. **Evidence-gated claims.** Every prompt that produced numbers included the instruction to compute them from the data and to report misses. This is why the classifier is reported at 84.96% (below the 85% target) and why the forecast section says a naive baseline won on the alumni-meet day.
5. **Verify with an independent harness.** After the UI was written we stopped trusting screenshots and wrote a DevTools driver that clicked every control and recorded console errors. It found a real bug (§5, P13) that eyeballing had missed.

---

3. Prompt log, A to Z

Legend for status: ✅ shipped · 🔧 shipped after correction · ⚠️ shipped with a known gap.

P00 — Project constitution (≈11:20) ✅

As typed (excerpt): "We are a student team at a 4-hour hackathon. HARD DEADLINE: 2:00 PM IST today... every number shown in the UI must be computed from the dataset. Never invent stats... Maintain docs/PROMPT_WORKBOOK.md... update after EVERY task."

Engineered prompt

> **Role.** You are the lead engineer on a four-hour hackathon build. **Objective.** Deliver a working, demonstrable sustainability product, judged from a 2:30 video on five equally weighted criteria (relevance, impact and scalability, use of AI and the dataset, technical feasibility, demo quality). **Hard rules.** (1) Any figure shown to a user must be computed from the supplied dataset, otherwise label it an assumption. (2) Keep the app runnable after every step; build vertically. (3) Raw files in `data/` are read-only; write derived data to `data/processed/`. (4) Provide a non-LLM fallback for any LLM feature. (5) Log every prompt, output and correction in the workbook, and never fill the human-contribution field on the team's behalf. **Process.** Phase 0 data audit → Phase 1 propose concepts and stop for a decision → Phase 2 build → Phase 3 demo assets. Warn if the time plan slips.

Output: `CLAUDE.md` (root), later extended with *Project State* and *Docs Index*. **Verification:** rules were re-read at each phase boundary; the "no invented numbers" rule is what caught the 85.0% rounding problem (P14). **Corrections:** the pasted brief still contained two placeholders (problem statement and dataset); the agent found the real problem statement already in `docs/` and used it instead of asking.

P01 — Data audit (≈11:22) ✅

As typed: the Phase 0 section of the brief: "Load every file in `data/`. Report: shape, columns + dtypes, nulls, value ranges, obvious quality issues, and 3–5 insights."

Engineered prompt

> Profile all seven `.xlsx` files. For each: shape, dtypes, null counts per column, value ranges, duplicate keys, and referential problems (aliases, units, late arrival). Then derive five insights that bear on the problem statement, each backed by a computed number, not a recollection. Write `docs/DATA_NOTES.md`. Do not modify `data/`; install any missing dependency in a project venv.

Output: `docs/DATA_NOTES.md`; venv with pandas, openpyxl, scikit-learn. **Verification and real findings:** 184 duplicate hourly rows (23,192 vs 23,040 expected); late arrivals up to 12.1 h; 37 spellings for 8 buildings; mixed units (kWh/units, L/kL, kg/bags); 13 free-text notes naming real faults; B05–B07 account for ~65% of electricity and ~95% of wet organic waste; solar covers 14.6%. **Corrections:** the first draft stated 63% and 93%. After recomputing with nulls excluded the agent corrected the doc to 65% and 95% in a later pass (P03).

P02 — Concept generation and the stop gate (≈11:30) ✅

As typed: “Propose 2–3 product concepts... Recommend one. STOP and wait for our pick.”

Engineered prompt

> Propose three product concepts. For each give a one-line pitch, how it uses each dataset file, the AI component, the quantified sustainability impact, and build risk. Recommend one and **stop**; do not write code until a human chooses.

Output: concepts A (full grid), B (classifier + forecaster only), C (incentive platform). Recommendation: A. **Verification:** the agent stopped as instructed. The choice was later *re-opened* by the ideation loop (P04), which is the point of having a gate.

P03 — Documentation backbone (≈11:35) 🔧

As typed: “Generate the project documentation. Hard limit: finish in 10 minutes... Mark every feature as ✅ Built, 🚧 In progress, or 📋 Planned, based on the actual code state. No invented numbers.”

Engineered prompt

> Produce PRD, development guide, architecture and dataflow (with valid Mermaid), and a problem-and-novelty document, each one to two screens. Status columns must reflect the actual repository state at the time of writing, not intent. Every figure is either computed from the dataset or labelled “Estimate (assumption: ...)”. Update the root `CLAUDE.md` in place without removing existing rules. Validate Mermaid syntax (quoted labels, no special characters). Commit.

Output: `docs/PRD.md`, `DEVELOPMENT.md`, `ARCHITECTURE_AND_DATAFLOW.md`, `PROBLEM_AND_NOVELTY.md`. **Verification:** statuses were set to 📋 Planned across the board because no code existed yet. **Corrections:** (1) `CLAUDE.md` did not exist on disk; it was created from the pasted brief. (2) The agent recomputed and fixed the 65%/95% figures from P01. (3) The agent flagged that the “picked” concept had never been confirmed and said so.

P04 — Idea selection by generator–critic loop (≈11:50 – 12:10) ✅

As typed (orchestrator): “You are the ORCHESTRATOR... Execute every step below in order without asking me questions. Do NOT write any product code... APPROVED = weighted score ≥ 8.5/10 AND no unresolved critical failure... Hard limit: 3 rounds.”

Two dedicated prompts live in the repo and are reproduced there in full: `prompts/ideator.md` and `prompts/evaluator.md`.

Engineered orchestrator prompt

> Coordinate an ideator and an evaluator as **separate, fresh** subagents; never pass the ideator’s reasoning to the judge. Round *N*: (a) ideator reads the problem, data notes and rules, plus the previous hand-off block, and writes `proposal_vN.md`; (b) evaluator scores it on the five rubric criteria, runs the critical-failure checks, answers skeptic questions and writes `eval_vN.md` with a copy-ready hand-off; (c) you read the evaluation, extract score and status, and either stop or feed the hand-off into round *N+1*. Maximum three rounds. If nothing reaches 8.5, report the best version at its **real** score and do not relabel it approved.

Ideator prompt design notes. Role: senior PM who has judged hackathons. Hard constraints: ~1.5 h build, Python only, dataset as the *engine*, no fabricated statistics, core value visible in 20 s. Forced output template of 16 sections, including a changelog that must answer every evaluator point. **Evaluator prompt design notes.** Role: independent judge who “did NOT create this proposal”. Calibrated bands (9–10 plausible winner, 8–8.9 strong), ten critical-failure checks (misread problem, decorative dataset, fabricated stats, missing LLM fallback, ...), skeptic questions, iteration tracking, and an instruction never to raise scores merely because something changed.

Results (real):

Round	Version	Weighted score	Status	What the judge forced
1	v1	**7.2**	Revision	The demo hook contradicted the data (B05 has no solar; wrong dates). Classifier only P1. Scope too large
2	v2	**8.0**	Revision	Hook rebuilt from verified events; classifier made guaranteed; cut lines. New gaps: retrospective cards, Drill leakage
3	v3	**8.3**	Revision (final)	Forward-looking card, leak-free Drill, named 30+ scale constraints

Verification: the judge checked claims against the data itself (e.g. confirmed `has_solar = 0` for B05; confirmed B07 consumed 1,143 kWh on 19 Sep against 580–660 on other weekends). **Outcome:** the product, **CircuGrid Brief**, was locked from v3 at 8.3/10. The gate was *not* passed and we said so (`docs/ideation/FINAL.md`).

P05 — Parallel plan, contracts and Git workflow (≈12:20) 🔧

As typed: "STOP THIS THING WE HAVE TO MAKE THE DEVELOPMENT INTO PHASES SUCH THAT ME AND MY TEAMMATE CAN WORK TOGETHER... phase wise branches and merge the branch phase wise."

Engineered prompt

> Stop implementation. Decompose the build into phases that two engineers can execute concurrently without blocking each other. Define, before any code, (1) file ownership per phase, (2) a contract document with function signatures and JSON schemas for every module boundary, (3) a branch-per-phase workflow with PR rules, sync points and a freeze time, (4) per-person prompt-log files to avoid merge conflicts. Update the PRD, CLAUDE.md and README to match, commit the shared base to `main` , push.

Output: `docs/PHASES.md` , `docs/CONTRACTS.md` , `docs/TEAM_WORKFLOW.md` , per-person workbook files; the cleaning pipeline committed as the shared base. **Corrections:** the agent had begun building sequentially; the interrupt redirected it. Later the team decided to build everything on `main` and not to merge the second developer's branch; the docs were updated to say so rather than left stale.

P06 — Data pipeline (≈12:15) ✅

As typed: (inside the build-plan prompt) "start building according to the build plan... start with the backend and ml first."

Engineered prompt

> Implement `src/pipeline.py` . (1) Deduplicate hourly rows on (timestamp, building) keeping the latest arrival. (2) Reindex to the full 2,880 h × 8 grid so offline buildings are explicit, never dropped; bridge only short gaps and keep a `missing` flag. (3) Map all building spellings to canonical IDs with a hand-curated alias dict; fail loudly on unmapped names. (4) Normalise units with a conversion table and reject values above stream-specific plausibility ceilings. (5) Convert the 13 free-text notes to causes via an ordered regex table. (6) Flag per-building robust-z outliers. (7) Persist to `data/processed/` and emit machine-readable stats.

Output: `src/pipeline.py` , `data/processed/*.csv` , `pipeline_stats.json` . **Verification (real):** 184 duplicates removed; 235 gap rows; 0 unresolved aliases across 37 spellings; 3 manual values rejected; 13 notes parsed; exactly one daily electricity outlier (B04, 27 Aug, 6,234 kWh) — which links to the "power analyzer being calibrated" note.

P07 — Forecast and root-cause attribution (≈12:35) 🔧

As typed: "start with my phases now... before pushing anything pull the latest code."

Engineered prompt

> **Forecast.** For each building and horizon (6 h, 12 h) fit a ridge regressor to predict hourly electricity from hour-of-day (sin/cos), day of week, day type, weather, event flag, current load, current footfall, same hour last week and same hour yesterday. Use a strict time-ordered split (train before 18 Sep). Score MAE and WAPE on three windows and against two naive baselines. Time one gradient-boosting fit for the accuracy-versus-compute argument. Report which model wins per building; **do not assume ridge wins**. **Attribution.** Detect anomaly episodes per building and stream against the median of days in the *same regime*. Attribute each episode in a fixed order: compatible same-building note within ±1 day → calendar event → regime shift → heavy rain → unexplained. Emit a ranked action card with an evidence line, quantified kWh/Rs/CO_2 where relevant, and confidence.

Output: `src/forecast.py` , `src/insights.py` ; PR #2. **Verification (real):** ridge best in 5 of 8 buildings at 6 h; last-week naive wins B04, B06, B08. On the alumni-meet day the yesterday baseline (WAPE 0.369) beat ridge (0.416). 10 episodes, all attributed. **Corrections:** the first attribution pass blamed solar dips on 9 Oct on a *calendar event*. The agent checked the calendar (30.8 mm rain) and changed the rule so solar skips events and regimes and is explained only by weather.

P08 — Outage Drill (≈12:55) 🔧

As typed: (same phase prompt) continue with my phases.

Engineered prompt

> Implement the Outage Drill: take a building offline for *N* hours using real data as hidden ground truth. Impute hidden hours from the **visible hours of the same day** plus a day-type hourly profile built from **training days only**. The

> headline estimate must not use any information from the hidden hours. Compare against: flat visible mean, same hours last week, unscaled profile, and — labelled separately as "best case once the daily summary arrives" — a daily-total-scaled variant. Provide a benchmark that averages error over every test day.

Output: `src/drill.py` ; PR #3. **Verification (real):** B04, 6 h window, 30 test days: mean error **1.37 kWh (4.2%)** vs 4.28 (last week), 4.76 (unscaled profile), 14.2 (flat). **Corrections:** (1) The branch imported a constant from a module that was not yet merged; the agent defined it locally to remove the cross-PR dependency. (2) The "best case" variant came out *worse* (1.83) than the headline. The agent kept the label but did not oversell it, and the README says so.

P09 — Merge, then fill the gaps (≈13:00) ✅

As typed: "OK SO MERGE ALL THE PRS" → "START BUILDING EVERYTHING AND MERGE DIRECTLY IN MAIN... we won't merge Jhashank's commits."

Engineered prompt

> Merge all open pull requests with squash. Inspect each before merging; resolve conflicts by preserving both sides' log entries. Then, on `main` , build the remaining backend: an edge bin classifier (stratified hold-out, per-event latency, routing), impact calculations with labelled assumptions, a baseline-relative incentive ranking, and a FastAPI service exposing each module, with a plain HTML page to verify the end-to-end flow. Pull before every push.

Output: `src/classifier.py` , `src/impact.py` , `src/incentive.py` , `src/api.py` , rough `web/index.html` . **Verification (real):** classifier **84.96%** on a stratified 20% hold-out (n = 1,263), ~5 ms per event. 5-fold CV 84.6%; extra trees and gradient boosting land at ~84%. **Corrections:** the agent noticed a PR from a third contributor (`NYN-05`) it had not authored, inspected the file list before merging, and flagged that it created a second, unreconciled cleaning pipeline. A merge conflict in the prompt log was resolved by hand.

P10 — "Is the data really clean?" (≈13:08) ✅

As typed: "tell me is the dataset used and cleaned and everything is working fine and is the data clean."

Engineered prompt

> Audit the processed data and report honestly: which of the seven files feed which module, residual nulls, duplicates, longest gap per building, unresolved aliases, rejected values, and whether the manual numeric readings agree with the meters. State what is *not* done.

Output: an audit table. Real findings: 0 duplicates left; longest gap 8 h (B07); daily nulls intentionally left as missing; manual electricity readings agree with the meter (median ratio 0.99, n = 152) but are **not used** as a gap-filler. **Value of the prompt:** it produced the first entry in the "known gaps" list now in the README.

P11 — Design-system port from a prior project (≈13:10) ✅

As typed: "OK SO WE WILL BE USING THE SAME FRONTEND THAT WE USED IN DIGIVAULT-BFB" (after an earlier message whose link did not arrive).

Engineered prompt

> Locate the DigiVault-BFB frontend in the local workspace. Extract its design tokens (ink/paper palette, single blue accent, Inter and mono type, lit-pill buttons, card shadows), the workspace shell (sticky header, 220 px sidebar), and the draggable wired-card graph (SVG cubic wires with glow, draggable cards, port dragging, hub that "seals" when all inputs connect, outputs that light up in sequence, stacked list on phones). Port it to framework-free HTML/CSS/JS served by the existing FastAPI app, rebinding every card to our real API data. No build step.

Output: `web/style.css` , `common.js` (graph engine), `workspace.js` ; later `app.html` . **Verification:** headless-Chrome screenshots of every view; no console errors. **Corrections:** (1) the agent first said the link was missing and asked for it instead of guessing the design. (2) A shell-expansion mistake turned `$('gAll')` into an empty string in an appended line and broke the script; the browser console caught it, the agent fixed it. (3) 84.96% was displaying as "85.0%" at one decimal, implying the target was met; changed to two decimals everywhere.

P12 — Landing page to a reference template (≈13:20) ✅

As typed: "https://ui.aceternity.com/pages/inference-landing-page OK SO THIS SHOULD BE THE LANDING PAGE... professional... the node diagram will be same."

Engineered prompt

> Fetch the reference template and extract its section order and interaction patterns. Recreate that structure for CircuGrid in vanilla HTML/CSS/JS: nav, tilt-in dashboard hero, tabbed features, the existing wired graph, a dot-map section, a developer panel with live responses, a catalogue tabs section, a comparison table, a closing call-to-action and footer. **Do not copy the template's copy, and do not invent users, testimonials or prices.** Every number on the page must be fetched from the API. Move the workspace to `/app`; refactor the graph into a shared module used by both pages.

Output: `web/index.html`, `landing.css`, `landing.js`; `/` landing, `/app` workspace; `/api/buildings`. **Verification:** desktop and 390 px mobile screenshots; the graph on the landing page and in the workspace share one implementation. **Corrections:** the fetch tool returned a text summary, not pixels, so the agent said it was matching *structure and feel* rather than claiming a pixel match. The template's "pricing" section was replaced by a status table ("Built / Partly / Design only") and a clearly labelled assumption table, because invented prices would breach the project rules.

P13 — Overview dashboard and hardening (≈13:40) 🔧

As typed: "make the dashboard and harden the frontend... should just wow... visually appealing." Then: "the dashboard component that is not working make it work because most of the section looks hard coded."

Engineered prompt (dashboard)

> Design a decision-grade overview. Requirements: animated KPI cards with sparklines; a stacked daily series for electricity, water, solar, landfill and composted with building toggles and a crosshair tooltip; staff notes and calendar events as pins on the series; a building-by-day anomaly heatmap normalised by day type so term/break shifts do not drown real anomalies; a ranked attention list with evidence; eight building cards; waste flow per building; accuracy ring; forecast mini-chart. All values from the API. Add backend endpoints rather than hard-coding any series.

Engineered prompt (hardening)

> Treat the UI as untrusted-input territory: HTML-escape every dataset string; show recoverable error states with a retry; use skeleton loaders; never hide content behind a delayed animation; respect reduced-motion; visible focus; sensible breakpoints. Warm slow endpoints in the background, gzip responses, add a health check. Then **test with a real browser driver**: click every control and report errors.

Output: `web/dashboard.js/.css`; `/api/timeseries`, `/api/health`, startup cache warm-up, gzip. **Verification (real):** a DevTools-protocol harness clicked all 7 views, 5 metric tabs, building toggles, chart and heatmap hovers, graph connect-all, drill, and navigation: **0 JavaScript errors** (one favicon 404, later fixed). **Corrections and honest finding.** The harness showed nothing was *broken*; the complaint was real in a different sense — everything *felt* hard-coded because every card opened the same page and Forecast/Brief had no selectors. We therefore added: a per-building drawer (opened from cards, heatmap rows and attention items), a Forecast explorer (any building × horizon × window, new `/api/forecast/{building}`), and Brief filters. Re-tested: 0 errors. A first screenshot also exposed staggered fade-ins leaving cards invisible in a captured frame; we removed the delays so content can never be hidden by animation.

P14 — Landing page: SaaS sections, and a bug the test found (≈14:30) 🔧

As typed: "make the landing page according to the given reference... our SaaS product... visually appealing and professional."

Engineered prompt

> Extend the landing page with the sections a product site needs: a live statistics strip, a three-step explanation, deployment plans, an FAQ, a fuller footer. Plans may state **hardware cost computed from the stated per-building assumption** and must say subscription pricing is not set. FAQ answers that quote metrics must read them from the API so they cannot drift. Verify by driving the page.

Output: stats strip, "how it works", plans (Rs 36,000 / 96,000 / 3,60,000 = buildings × Rs 12,000, assumption), live-data FAQ, footer. **Corrections:** the driver reported the new sections rendering unstyled. Root cause: a `<section id="plans">` and an inner `<div id="plans">` shared an id, so `$('plans').innerHTML = …` replaced the entire section including its container. Fixed by renaming the inner elements (`planGrid`, `faqList`).

P15 — Ship it (≈14:35) ✅

As typed: "push every thing on github and we will deploy it" ... "do fast push every thing."

Engineered prompt

> Make the repository deployable from a clean checkout: location-independent file access, a container image, a process-file start command, a PaaS manifest with a health check, deployment notes in the README. Confirm the working tree and remote are in sync. Skip the container build test if time-boxed, but say that it was not run.

Output: `Dockerfile`, `Procfile`, `render.yaml`, README deploy section; two modules switched from a CWD-relative `data/calendar.xlsx` path to an absolute one. **Verification:** imports pass; working tree clean and equal to `origin/main`. **The Docker image has not been built or run** (stated in the hand-off).

P16 — README and this workbook (≈14:45) ✅

As typed: "make a very professional readme... and push it to github" → "make the prompt work book md file in docs... for the entire project... very technical."

Engineered prompt (README)

> Write the repository front page for a technical reviewer: problem, value, features mapped to files, results with how each was measured, architecture diagram, data and cleaning rationale, model details, a complete API table with curl, quick start, deployment, repository layout, a constraint-by-constraint status table, and an explicit assumptions-and-limitations section. Use only figures that the code reproduces. Add screenshots.

Output: `README.md`, `docs/screenshots/`. **Engineered prompt (this workbook):** produce a technical log of the whole build with original wording preserved beside refined prompts, and an honest account of failures. **Corrections:** the agent proposed keeping original wording next to the refined prompt rather than presenting reconstructed prompts as verbatim, because the workbook is a judged record of process.

---

4. Architecture decisions that came out of prompting

Decision	Prompted by	Rationale
FastAPI + framework-free frontend	P05 stack change, P11	One process, no build step, same API for rough and final UI
Ridge per building, not a deep model	P07 + problem statement's edge constraint	Fits in ~10 ms, edge-sized; gradient boosting fit was timed for the trade-off
Strict time split at 18 Sep	P07	Test window contains both the alumni meet and the exam stretch
Drill headline never sees hidden hours	P04 evaluator ("circular")	The judge pointed out the daily-total leak; the daily-total bar is shown separately, labelled
Attribution order note → event → regime → weather	P07	Most specific evidence first; solar excluded from events
Sensor-fault class (B04 27 Aug)	P04 evaluator, P06	A big reading is not necessarily waste; excluded from savings
`data/processed` git-ignored, rebuilt on start	P05, P15	No merge conflicts, reproducible

---

5. Iteration ledger (what went wrong and how it was caught)

#	Defect	Introduced in	Caught by	Fix
1	Demo hook cited B05 solar (B05 has none) and mis-dated events	Ideator v1	Evaluator, against the data	v2 rebuilt from verified events
2	Drill error leaked the hidden day total	Ideator v2	Evaluator	Headline uses visible hours only; daily-total bar labelled "best case"
3	Share of electricity quoted as 63% / 93%	P01	Agent recompute	65% / 95%
4	Solar dips on 9 Oct attributed to a calendar event	P07	Agent cross-check with weather (30.8 mm rain)	Solar attributes to weather only
5	Cross-branch import broke the Drill branch	P08	Run-time error	Local constant
6	Merge conflict in the workbook log	P09	Git	Kept both sections
7	Accuracy rendered "85.0%"	P11	Rule "no invented/rounded-up numbers"	Two decimals everywhere

#	Defect	Introduced in	Caught by	Fix
8	Appended JS line broken by shell expansion	P11	Browser console	Rewritten with a quoted heredoc
9	Cards invisible mid-animation in captures	P13	Screenshot review	Removed animation delays
10	Dashboard felt hard-coded	P13	User	Drawer, forecast explorer, filters
11	Duplicate element ids wiped two landing sections	P14	DevTools driver	Renamed inner ids
12	Favicon 404	P13	Driver	Inline SVG favicon

6. Verified results and honest misses

All values come from running this repository.

Quantity	Value	Note
Hourly duplicates removed	184	23,192 → 23,040 grid rows
Gap rows flagged	235	longest gap 8 h
Building spellings resolved	37 → 8	0 unresolved
Notes parsed into causes	13	
Anomaly episodes	10 attributed, 0 unexplained	
Forecast, 6 h, all test days	ridge best in 5/8 buildings	last-week naive wins B04, B06, B08
Alumni meet (B07, 19 Sep)	yesterday-naive 0.369 beats ridge 0.416	reported as is
Drill, B04, 6 h	1.37 kWh mean error vs 4.28 / 4.76 / 14.2	30 test days
Classifier hold-out accuracy	**84.96%**	**Target was >85%: missed by 0.04 pt**
Classifier latency	≈ 5 ms per event	laptop CPU, not edge hardware
Campus baseline	750,969 kWh ≈ 525.7 t CO_2 ≈ Rs 6.0 M	includes the flagged 6,234 kWh fault; charts show it at median

What is not done: manual meter readings are not fused; the classifier has not run on edge hardware; no live buffering hardware (the Drill runs on recorded data); the incentive loop is a baseline ranking with no behaviour change measured; the optional LLM text path was never built; the container image was not build-tested; the ideation loop never reached its own 8.5 gate (best 8.3).

7. Reproducing the work

```
 git clone https://github.com/HARDIKSINGH150206/p2p.git && cd p2p
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m src.pipeline && uvicorn src.api:app --port 8000
# landing: http://localhost:8000   workspace: http://localhost:8000/app
```

Idea-selection record: `prompts/ideator.md`, `prompts/evaluator.md`, `docs/ideation/proposal_v1–v3.md`, `eval_v1–v3.md`, `FINAL.md`.

8. Human contribution

The agent does not fill this section. Team members record what they decided, edited or rejected for each phase.

Phase	Decision / edit / rejection
P00 Constitution	TODO (team):
P01–P02 Data audit and concept choice	TODO (team):
P03 Documentation	TODO (team):
P04 Ideation loop (which version to build from, and why not to re-run)	TODO (team):
P05 Parallel plan; decision to build on `main` and not merge the second developer's branch	TODO (team):

Phase	Decision / edit / rejection
P06–P09 Pipeline, models, API	TODO (team):
P10 Data-quality review	TODO (team):
P11 Choice of the DigiVault design system	TODO (team):
P12 Choice of the reference template	TODO (team):
P13–P14 Dashboard and landing review	TODO (team):
P15–P16 Deployment, README, workbook	TODO (team):

Per-person raw logs: `docs/workbook/hardik.md`, `docs/workbook/jhashank.md`, earlier running log `docs/workbook/_legacy_running_log.md`.